TRENDVIBE ATELIER

Enterprise Java EE Full-Stack E-Commerce • Master Interview Blueprint

Candidate Role: Full-Stack Java Developer	**Target Stack:** Java EE, Servlets, JDBC, MySQL, HTML5/ES6
Project Scope: 535-Piece E-Commerce System	**Architecture:** 3-Tier Enterprise MVC & DAO Pattern

1. Languages & Technology Stack (Pin-to-Pin Breakdown)

When an interviewer asks: *'What languages and technologies did you use in this project, and why?'*, give this structured, confident explanation covering all 4 tiers of the system:

Tier	Language / Technology	Role & Engineering Justification
Backend Core	**Java 17 (Java EE)** Servlets 5.0, JSP, JSTL	Handles HTTP routing, business logic, session validation, and thread-safe controller orchestration via the MVC architecture.
Persistence	**SQL & JDBC** MySQL 8.0, HikariCP	Relational database normalized to 3NF, parameterized PreparedStatements to eliminate SQL injection, and high-speed connection pooling.
Frontend UI	**HTML5, CSS3, JavaScript (ES6+)**	Clean Zara-inspired atelier interface. Mobile viewport optimization (58px header, touch rails), lazy loading, and error fallback handlers.
Server & Ops	**Apache Tomcat 10, Maven, Docker, Git**	Tomcat manages servlet worker threads. Maven handles dependencies (`pom.xml`). Docker containerizes deployment. GitHub Pages hosts client demo.

2. How to Introduce Your Project (90-Second Pitch)

Interviewer Question: ***'Tell me about your project and your primary contributions.'***

Your Exact Answer to Speak:

*"I engineered **TrendVibe Atelier**, a high-performance, enterprise-grade full-stack e-commerce web application designed for luxury retail. The architecture is built on a 3-tier MVC model using **Java EE Servlets** on the backend, **MySQL 8.0** managed via **JDBC & HikariCP** for persistence, and a modern, responsive **HTML5/CSS3/Vanilla JavaScript** frontend.*

My key contributions include:
*1. **Database Engineering:** Designed a normalized 8-table relational schema in 3NF, ensuring ACID compliance and zero race conditions during checkout using pessimistic row-level locking (`SELECT ... FOR UPDATE`).*
*2. **Backend Architecture:** Built thread-safe Java Servlets handling customer registration, phone+password authentication, dynamic catalog filtering across 535 products, and cart state management.*
*3. **Performance & Mobile Optimization:** Solved mobile viewport overflow with a responsive 58px header, added client-side image fallback handlers to eliminate broken image rendering, and optimized pagination to render 24 items per batch with sub-100ms response time."*

3. Step-by-Step Database Creation Guide (How to Create the DB)

When an interviewer asks: *'How did you create the database? Explain the tables, keys, and queries step-by-step.'*, explain this exact technical sequence:

Step 1: Database Initialization & Character Encoding

We create a dedicated schema with utf8mb4 character set to support international characters, currencies, and emojis:

```
CREATE DATABASE ecommerce_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE ecommerce_db;
```

Step 2: Designing the 8 Normalized Tables (3NF Schema)

The database is normalized to Third Normal Form (3NF) to eliminate transitive dependencies and data anomalies. Below are the core tables and constraints:

```
-- 1. USERS TABLE: Customer accounts with phone + password credentials
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    pincode VARCHAR(10),
    city VARCHAR(50),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. CATEGORIES TABLE: Master product taxonomy
CREATE TABLE categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL UNIQUE,
    description TEXT,
    is_active BOOLEAN DEFAULT TRUE
);
```

```
-- 3. PRODUCTS TABLE: Master luxury catalog (535 pieces)
CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    name VARCHAR(200) NOT NULL,
    brand VARCHAR(100) NOT NULL,
    gender ENUM('Men', 'Women', 'Unisex') DEFAULT 'Unisex',
    price DECIMAL(10, 2) NOT NULL,
    original_price DECIMAL(10, 2) NOT NULL,
    discount_percent INT DEFAULT 0,
    image_url VARCHAR(500) NOT NULL,
    rating DECIMAL(2, 1) DEFAULT 4.5,
    reviews_count INT DEFAULT 0,
    FOREIGN KEY (category_id) REFERENCES categories(category_id) ON DELETE RESTRICT
);

-- 4. INVENTORY TABLE: Strict row-level stock management
CREATE TABLE inventory (
    inventory_id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT NOT NULL UNIQUE,
    stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
    warehouse_code VARCHAR(20) DEFAULT 'WH-BLR-01',
    last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE CASCADE
);

-- 5. ORDERS TABLE: Order header and financial summary
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    total_amount DECIMAL(10, 2) NOT NULL,
    shipping_address TEXT NOT NULL,
    shipping_pincode VARCHAR(10) NOT NULL,
    order_status ENUM('PENDING', 'CONFIRMED', 'DISPATCHED', 'DELIVERED', 'CANCELLED') DEFAULT 'CONFIRM
ED',
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
);

-- 6. ORDER_ITEMS TABLE: Itemized line records with snapshot pricing
CREATE TABLE order_items (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    selected_size VARCHAR(10) NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10, 2) NOT NULL, -- Historical price snapshot
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- 7. PAYMENTS TABLE: Financial transaction audits
CREATE TABLE payments (
    payment_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    payment_mode ENUM('UPI', 'CARD', 'NETBANKING', 'COD') NOT NULL,
    transaction_ref VARCHAR(100) UNIQUE,
    payment_status ENUM('SUCCESS', 'PENDING', 'FAILED') DEFAULT 'SUCCESS',
```

```
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);

-- 8. INDEXING FOR HIGH PERFORMANCE
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_products_price ON products(price);
CREATE INDEX idx_orders_user ON orders(user_id);
```

4. Architecture, Concurrency & Thread-Safety

A. The 3-Tier Enterprise MVC Architecture

• **Presentation Tier:** Client-side DOM manipulated using Vanilla JavaScript (ES6). Renders dynamic product cards, Quick View drawer, and handles image fallbacks.
• **Controller & Service Tier:** Java Servlets (`ProductServlet`, `AuthServlet`, `OrderServlet`) handle incoming `HttpServletRequest` objects, validate inputs, invoke business logic, and produce JSON responses or dispatch to JSPs.
• **Data Access Tier (DAO Pattern):** Encapsulates all SQL logic inside DAO classes (`ProductDAO`, `UserDAO`, `OrderDAO`). Uses `PreparedStatement` to prevent SQL injection and `HikariCP` for high-throughput connection pooling.

B. Concurrency Locking During Checkout (Preventing Overselling)

Problem: If two customers attempt to purchase the last available item simultaneously, a race condition occurs.
Solution: We use **Pessimistic Locking** with ACID transaction isolation:

```
connection.setAutoCommit(false); // Begin ACID Transaction
PreparedStatement lockStmt = connection.prepareStatement(
    "SELECT stock_quantity FROM inventory WHERE product_id = ? FOR UPDATE"
);
lockStmt.setInt(1, productId);
ResultSet rs = lockStmt.executeQuery();
if (rs.next() && rs.getInt("stock_quantity") >= orderQty) {
    // Deduct stock and insert order record
    connection.commit();
} else {
    connection.rollback(); // Insufficient inventory
}
```

5. Top 35+ Technical Interview Questions & Word-for-Word Answers

Q1: How did you apply Object-Oriented Programming (OOP) in this project?

We applied all 4 pillars of OOP: 1) Encapsulation in Model classes (Product, User, Order) using private fields and public getters/setters. 2) Abstraction through DAO interfaces (e.g., ProductDAO defining findById without exposing SQL details). 3) Inheritance where custom servlets extend HttpServlet. 4) Polymorphism via interface-driven service contracts.

Q2: Why did you use PreparedStatement instead of Statement in JDBC?

Two vital reasons: 1) Security: PreparedStatement uses parameterized queries, pre-compiling the SQL statement in the database engine and treating user input strictly as literal values, completely neutralizing SQL Injection attacks. 2) Performance: Pre-compiled statements are cached in the database execution plan cache, speeding up repeated queries.

Q3: Are Java Servlets thread-safe? How do you ensure safety in your servlets?

No, by default a Servlet instance is a singleton created once by the Tomcat container. Multiple client requests are handled by different threads invoking service(), doGet(), or doPost() on that same instance simultaneously. Therefore, defining instance (class-level) variables creates race conditions. We ensure thread-safety by keeping servlets completely stateless—declaring variables strictly inside request handler methods (local stack-allocated variables).

Q4: Explain the complete Servlet Lifecycle.

1. Loading & Instantiation: Tomcat loads the servlet class and instantiates it. 2. Initialization: Tomcat calls init(ServletConfig config) once to allocate resources. 3. Request Handling: For every incoming HTTP request, Tomcat spawns a thread that calls service(), which routes to doGet() or doPost(). 4. Destruction: When the web app shuts down, Tomcat calls destroy() to release connections and resources.

Q5: What is the difference between doGet() and doPost()?

doGet() is used to retrieve data without altering server state; parameters are appended in the URL query string (visible in browser history, max length limited ~2KB, bookmarkable, cached). doPost() sends data in the HTTP request body; it is used for state-altering operations like user registration, placing orders, and password updates (secure, no payload size limitation, not cached).

Q6: How do you handle database connection management in production?

Instead of opening and closing physical database sockets on every HTTP request using DriverManager (which causes severe latency and socket exhaustion), we implement a connection pool using HikariCP. HikariCP keeps a warm pool of pre-established database connections. Servlets borrow a connection via try-with-resources and return it to the pool in milliseconds.

Q7: What is the optimal connection pool sizing formula for HikariCP?

The standard PostgreSQL/HikariCP formula is: connections = (core_count * 2) + effective_spindle_count. For a 4-core database server with SSD storage, a pool size of (4 * 2) + 1 = 9 to 10 connections comfortably serves thousands of concurrent users with zero queue contention.

Q8: What are ACID properties and how did you implement them?

A - Atomicity: Either all order items and payment records are saved, or none are. Handled via connection.setAutoCommit(false), connection.commit(), and connection.rollback() inside catch blocks. C - Consistency: Database constraints (foreign keys, CHECK stock_quantity >= 0) ensure invalid state cannot be saved. I - Isolation: Row-level locks prevent dirty reads. D - Durability: MySQL InnoDB write-ahead redo logs guarantee committed data survives crashes.

Q9: Why is unit_price duplicated in order_items if products table already has price?

This is intentional historical snapshotting. If an item sells today for ■2,499 and the merchant updates its catalog price next month to ■2,999, past order receipts and accounting ledgers must still show the original purchase price of ■2,499. Reading from products table would corrupt historical transaction records.

Q10: What is Third Normal Form (3NF) and why is it important?

A table is in 3NF if: 1) It is in 2NF (no partial dependencies on a composite primary key), and 2) It has no transitive dependencies (non-key columns must depend strictly on the primary key, and nothing else). This eliminates duplicate data, saves storage, and prevents insert, update, and delete anomalies.

Q11: Why did you choose Phone Number + Password over SMS OTP for customer login?

SMS OTP introduces third-party SMS gateway latency, carrier dropouts, recurring per-SMS billing costs, and SIM-swap attack vulnerabilities. Phone Number + Password provides instant, zero-latency authentication, offline testability, and standard enterprise security backed by BCrypt hashing and rate-limiting.

Q12: How do you securely store passwords in the database?

We never store plaintext passwords. We use the BCrypt adaptive hashing algorithm with a random salt (cost factor 12). BCrypt is intentionally slow and CPU-intensive, rendering rainbow table lookups and brute-force GPU attacks mathematically infeasible.

Q13: How does session management work between the browser and Tomcat?

When a user signs in, Tomcat generates a unique 128-bit cryptographically secure session ID, stores the session object in memory (`HttpSession`), and sends the ID to the client via an HTTP cookie named `JSESSIONID`. The cookie is marked `HttpOnly` (inaccessible to malicious JavaScript XSS) and `SameSite=Strict` (prevents Cross-Site Request Forgery).

Q14: How does the application prevent Cross-Site Scripting (XSS)?

All user-generated text inputs (names, review comments, product searches) are sanitized and HTML-escaped before being rendered onto the DOM (replacing < with <, > with >). This prevents attackers from injecting <script> tags into product reviews or customer profiles.

Q15: How did you fix the issue where images appeared blurred or broken with a blue question mark?

We diagnosed two root causes: 1) Several Unsplash image IDs were deleted, returning HTTP 404. 2) Without an `onerror` attribute, the browser rendered its default broken image box. We resolved this by verifying all photo URLs against HTTP 200 OK and implementing an automatic JavaScript error handler `onerror="handleImageFallback(this, category)"` which instantly swaps any failing image to a guaranteed high-res category editorial image.

Q16: How did you fix mobile layout overflow issues?

We implemented: 1) Viewport-fit cover with a locked 58px mobile header, converting text buttons into SVG icons. 2) Touch-friendly single-row horizontal category rail with `-webkit-overflow-scrolling: touch`. 3) 24-item batch pagination to avoid loading all 535 DOM nodes simultaneously, maintaining 60 FPS mobile scroll performance.

Q17: What happens when two users place an order for the last item simultaneously?

The first user's transaction acquires a row-level pessimistic lock via `SELECT ... FOR UPDATE` on the inventory table. The second user's thread waits until the first transaction commits. When the lock releases, the second user's query reads `stock_quantity = 0`, triggers an out-of-stock exception, and rolls back cleanly with a friendly error message.